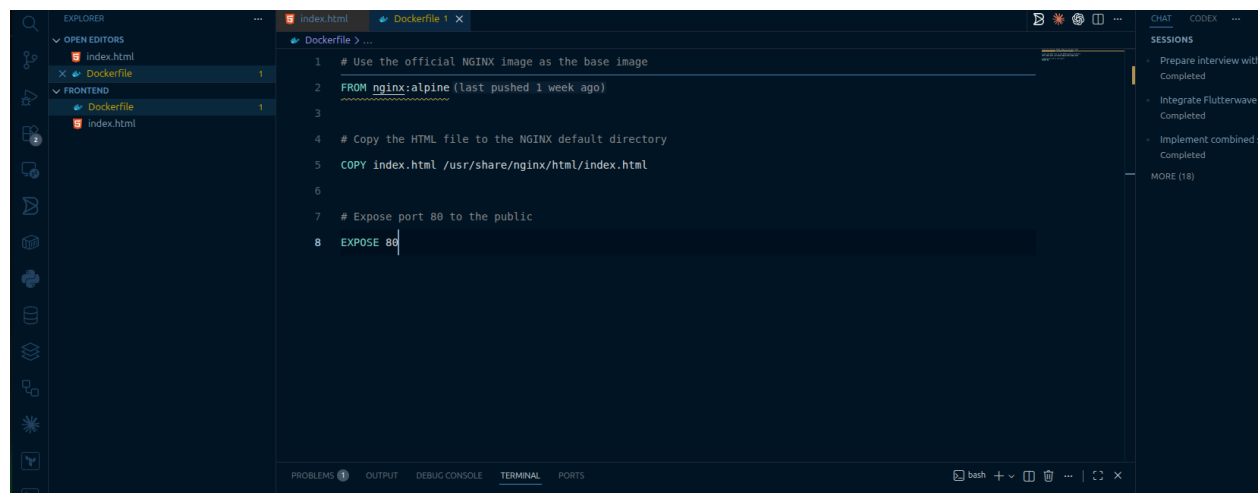


Project Structure Setup

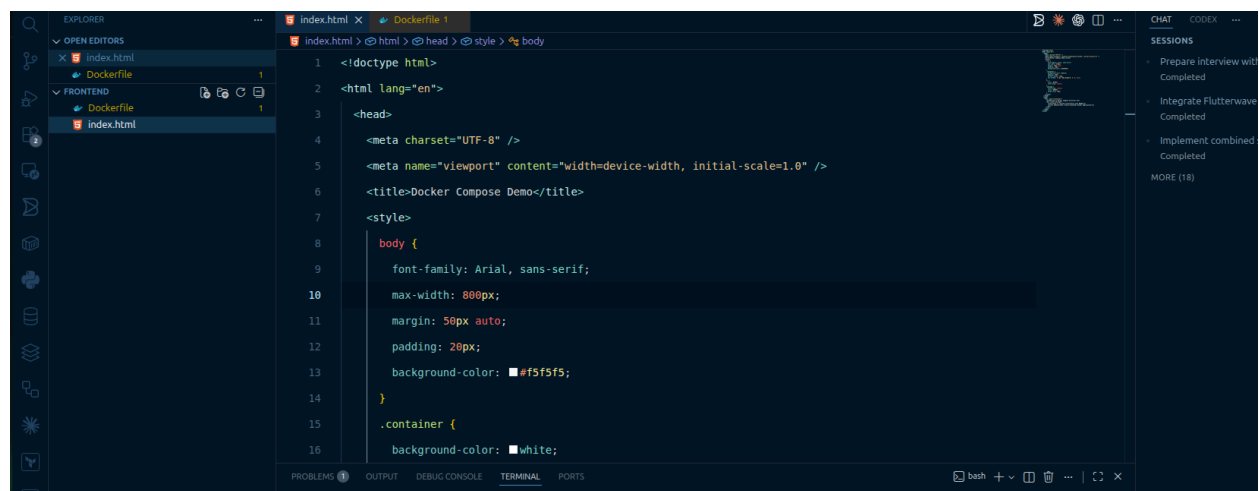
```
richie@richie-Latitude-5490:~/Azubi Africa/Group Activities$ mkdir docker-compose-activity
richie@richie-Latitude-5490:~/Azubi Africa/Group Activities$ cd docker-compose-activity
richie@richie-Latitude-5490:~/Azubi Africa/Group Activities/docker-compose-activity$ mkdir frontend backend && touch docker-compose.yml
richie@richie-Latitude-5490:~/Azubi Africa/Group Activities/docker-compose-activity$ ls
backend  docker-compose.yml  frontend
richie@richie-Latitude-5490:~/Azubi Africa/Group Activities/docker-compose-activity$
```

Frontend Index.html and Docker files



This screenshot shows the VS Code interface with the Dockerfile for the frontend service open in the editor. The Explorer sidebar on the left shows the project structure with 'index.html' and 'Dockerfile' files under the 'FRONTEND' directory. The Dockerfile content is as follows:

```
1 # Use the official NGINX image as the base image
2 FROM nginx:alpine (last pushed 1 week ago)
3
4 # Copy the HTML file to the NGINX default directory
5 COPY index.html /usr/share/nginx/html/index.html
6
7 # Expose port 80 to the public
8 EXPOSE 80
```



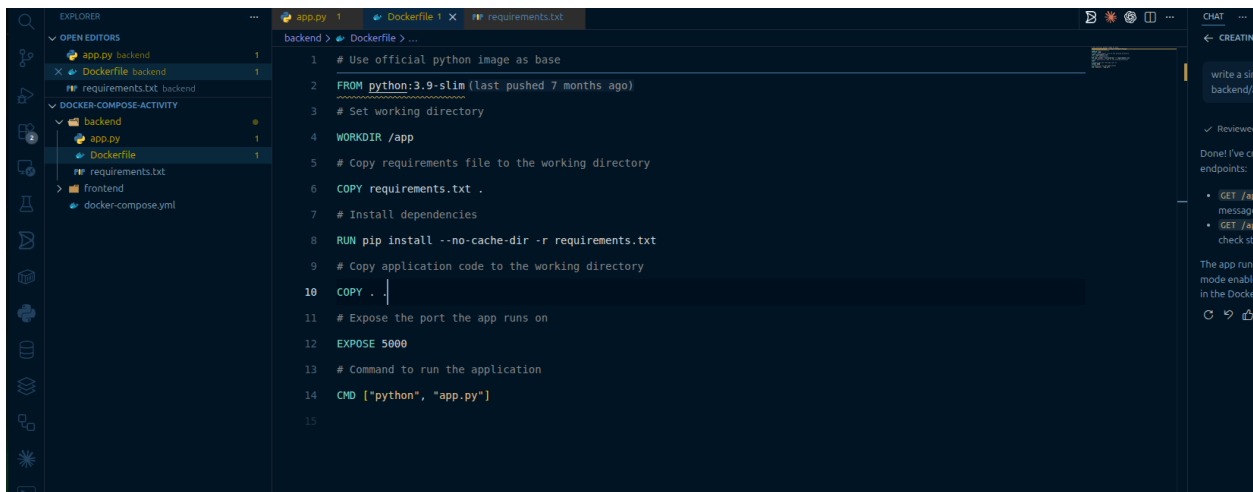
This screenshot shows the VS Code interface with the index.html file for the frontend service open in the editor. The Explorer sidebar on the left shows the project structure with 'index.html' and 'Dockerfile' files under the 'FRONTEND' directory. The index.html content is as follows:

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Docker Compose Demo</title>
7     <style>
8       body {
9         font-family: Arial, sans-serif;
10        max-width: 800px;
11        margin: 50px auto;
12        padding: 20px;
13        background-color: #f5f5f5;
14      }
15      .container {
16        background-color: #white;
```

Backend API [app.py](#) and docker files

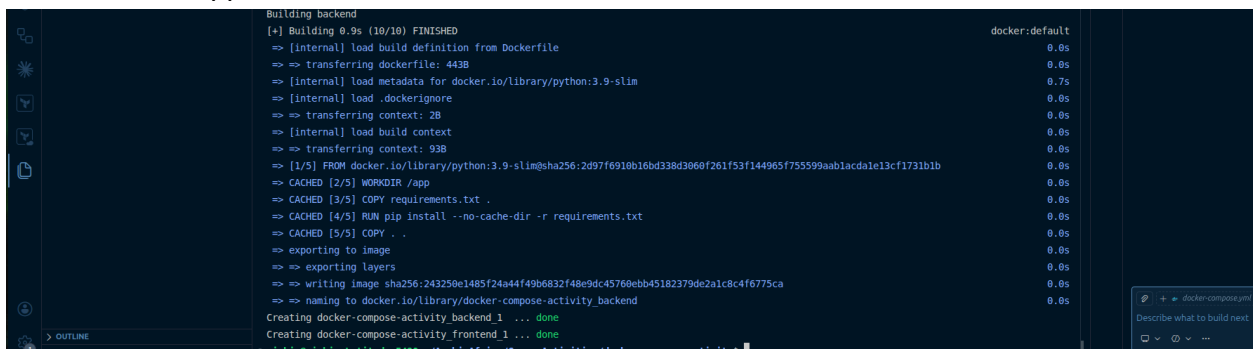


```
1 from flask import Flask, jsonify
2
3 app = Flask(__name__)
4
5 @app.route('/api/message', methods=['GET'])
6 def get_message():
7     return jsonify({"message": "Hello from the Flask API!"})
8
9 @app.route('/api/health', methods=['GET'])
10 def health_check():
11     return jsonify({"status": "healthy"}), 200
12
13 if __name__ == '__main__':
14     app.run(host='0.0.0.0', port=5000, debug=True)
15
```



```
1 # Use official python image as base
2 FROM python:3.9-slim (last pushed 7 months ago)
3 # Set working directory
4 WORKDIR /app
5 # Copy requirements file to the working directory
6 COPY requirements.txt .
7 # Install dependencies
8 RUN pip install --no-cache-dir -r requirements.txt
9 # Copy application code to the working directory
10 COPY . .
11 # Expose the port the app runs on
12 EXPOSE 5000
13 # Command to run the application
14 CMD ["python", "app.py"]
15
```

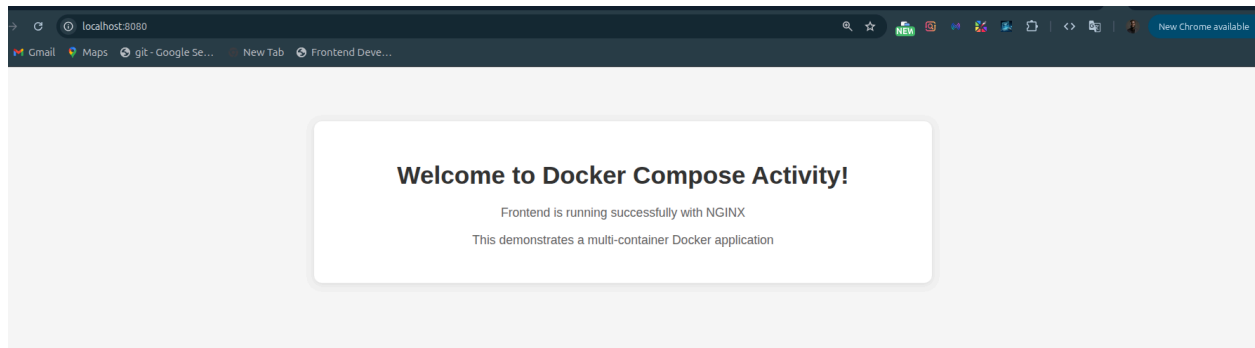
Run and build Applications



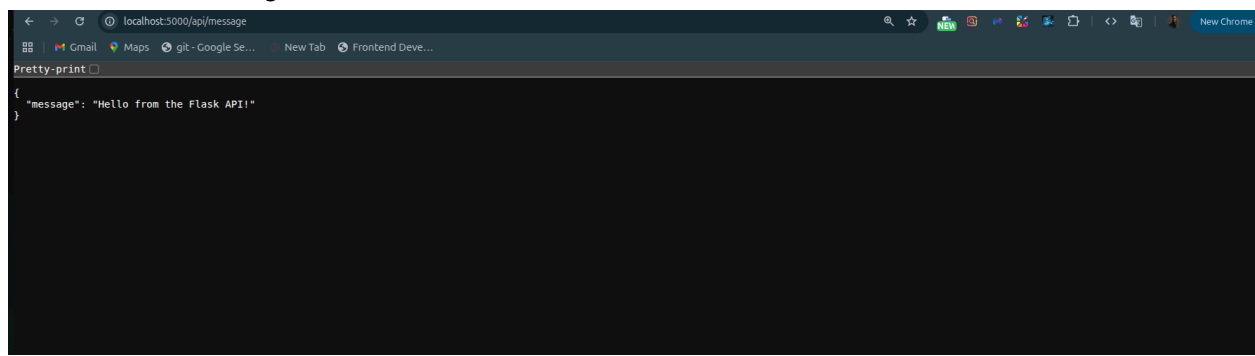
```
Building backend
[+] Building 0.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.7s
=> [internal] load .dockerignore 0.0s
=> [internal] load build context 0.0s
=> [internal] transferring context: 93B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599a0b1acda1e13cf1731b1b 0.0s
=> [2/5] WORKDIR /app 0.0s
=> [3/5] COPY requirements.txt . 0.0s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [5/5] COPY . . 0.0s
=> exporting to image 0.0s
=> exporting layers 0.0s
=> writing image sha256:243259e1485f24a44f49b6832f48e9dc45760ebb45182379de2a1c8c4f6775ca 0.0s
=> naming to docker.io/library/docker-compose-activity-backend 0.0s
Creating docker-compose-activity-backend 1 ... done
Creating docker-compose-activity-frontend 1 ... done
richie@richie-latitude-5498:~/Africa/Groom_Activities/docker-compose-activities
```

Test Application

Frontend running on 8080



Backend API running on 5000



Explore Docker Compose commands:

List services:



Stop services:



View logs:

```
richie@richie-Latitude-5490:~/Azubi Africa/Group Activities/docker-compose-activity$ docker-compose logs
Attaching to docker-compose-activity_frontend_1, docker-compose-activity_backend_1

backend_1 | * Serving Flask app 'app'
backend_1 | * Debug mode: on
backend_1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
backend_1 | * Running on all addresses (0.0.0.0)
backend_1 | * Running on http://127.0.0.1:5000
backend_1 | * Running on http://172.24.0.2:5000
backend_1 | Press CTRL+C to quit
backend_1 | * Restarting with stat
backend_1 | * Debugger is active!
backend_1 | * Debugger PIN: 978-165-601
backend_1 | 172.24.0.1 - - [05/Jun/2026 11:04:55] "GET / HTTP/1.1" 404 -
backend_1 | 172.24.0.1 - - [05/Jun/2026 11:04:56] "GET /favicon.ico HTTP/1.1" 404 -
backend_1 | 172.24.0.1 - - [05/Jun/2026 11:07:42] "GET /api/message HTTP/1.1" 200 -
frontend_1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend_1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend_1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontend_1 | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
frontend_1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend_1 | /docker-entrypoint.sh: Configuration complete; ready for start up
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: using the "epoll" event method
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: nginx/1.31.1
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: OS: Linux 6.17.0-35-generic
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: getrlimit(RLIMIT_NOFILE)= 1024:524288
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: start worker process 20
frontend_1 | 2026/06/05 10:47:01 [notice] 1#1: start worker process 21
```